Adaptive Sketching for Open-Leg Tensor Network Contraction

Kaiyu Shen

Abstract

We study tensor network contraction with open legs when the task is to explicitly materialize the full output tensor rather than a scalar or a small set of local queries. In this setting, the output representation itself is fixed by the task, so the main opportunity for acceleration lies in reducing the internal contraction cost while keeping the dense output unchanged. We therefore propose AS-TNC, a framework that partitions the output index space into blocks, builds a binary partition tree over the interaction graph, and recursively propagates separator states along the tree. The method keeps the output boundary explicit, applies adaptive low-rank approximation only to internal interfaces, and uses implicit sketch-based merging to avoid constructing inflated intermediate interfaces. Across representative classical tensor networks, AS-TNC keeps the relative error below 4.81×10^{-3} while achieving speedups from 13.98× to 84.05× over direct exact contraction. Additional analysis shows that these gains come almost entirely from the contraction stage, that implicit sketch merging is the dominant efficiency source, and that approximation strength provides a stable knob for controlling the error-time trade-off.

1 Introduction

Tensor networks provide a structured representation for high-dimensional arrays and are widely used in quantum many-body simulation, statistical physics, graphical-model inference, and tensor algorithms [1]. A large body of work on tensor network contraction (TNC) has focused on three recurring questions: how to design better contraction orders, how to control the size of intermediate states, and how to reduce the cost of approximate contraction by compressing those intermediate states [2–9]. Most of this literature targets scalar-valued contraction, local observables, or local queries in probabilistic inference, rather than explicitly generating the full tensor that remains after preserving open legs [1, 7, 10, 11]. Once full materialization is required, both compute and memory costs can increase dramatically [4].

This paper studies precisely that regime: tensor network contraction with open legs when the final output must be emitted as a dense tensor. The key distinction from standard approximate contraction is that the algorithm cannot change the form of the result. It cannot replace the final answer with a compressed representation, nor can it return only a few partial observables. Instead, the goal is to reduce the cost of producing the same dense output. Writing the end-to-end time as

$$T_{\text{total}} = T_{\text{contract}} + T_{\text{emit}},$$

we interpret T_{contract} as the internal cost of contraction, merging, and intermediate-state processing before the final tensor is written out, and T_{emit} as the cost of materializing those results in the final output buffer. Because the dense output itself is mandatory, T_{emit} is usually unavoidable. The main algorithmic target is therefore T_{contract}.

Motivated by this observation, we develop an approximate contraction framework for full materialization. The central idea is to build a partition tree over the interaction graph, recursively maintain separator states for subnetworks, and keep the output boundary explicit throughout the computation. Approximation is applied only to separator states that propagate inside the network. In other words, we do not approximate the final output object itself; we approximate the internal states that must be traversed in order to produce that output. This perspective is what distinguishes our setting from methods that directly compress the output side and from methods designed for scalar contraction.

From a representation viewpoint, each subtree state can be interpreted as an interface matrix from output-side configurations to boundary-side configurations. We propagate these states on the tree in low-rank factored form and recompress them after each merge to control rank growth. To avoid the common bottleneck in which interface dimensions inflate before being compressed again, we further introduce implicit sketch-based merging: rather than explicitly forming the merged interface, we recover its dominant subspace through operator access. The result is a unified pipeline that combines partition-tree recursion, separator-state approximation, and sketch-driven recompression while preserving output semantics.

Our experiments address three questions. First, can AS-TNC provide a practically useful accuracy-time trade-off for full-materialization TNC? Second, do the observed gains genuinely come from the contraction stage rather than from output write-back artifacts? Third, how do approximation strength and key mechanisms affect behavior? On representative classical networks, AS-TNC keeps the relative error below 4.81×10^{-3} while achieving speedups from 13.98× to 84.05× over exact TNC. A fixed small-rank baseline is faster in some cases, but its relative error rises to 0.65–0.98. Additional analysis shows that

the end-to-end gains are almost entirely contraction-side, that implicit sketch merging is the most stable source of efficiency, and that the approximation level acts as a reliable control knob for error.

The main contributions of this paper are:

- We formulate approximate contraction for open-leg tensor networks under a full dense-materialization requirement, shifting the approximation target from the output tensor to the internal contraction process.
- We propose AS-TNC, which recursively maintains separator states on a partition tree, uses adaptive low-rank compression to control interface size, and employs implicit sketch-based merging to avoid explicitly forming inflated merged interfaces.
- We empirically validate the method on Ring-8, Tree-8, Grid 3×3, and Random-8, showing a clear accuracy-time trade-off together with interpretable evidence about where the gains come from.

2 Related Work

Exact tensor network contraction. Classical work on exact tensor network contraction has concentrated on contraction ordering, contraction trees, treewidth-aware planning, and slicing strategies. A consistent takeaway from this line of research is that the dominant difficulty is usually not the original tensors themselves, but the size of the intermediate states they induce and the resulting time and memory complexity [2–5]. Index slicing further shows that when a monolithic contraction is too expensive, one can cut selected degrees of freedom and solve many smaller subproblems in parallel or in batches [6]. These ideas directly motivate our setting: separator growth is the true bottleneck, and blockwise organization can be an important systems-level tool when full materialization is required. However, the primary objective of those methods is still exact contraction rather than approximate acceleration under an unchanged dense-output requirement.

Approximate contraction and intermediate-state compression. When exact contraction becomes prohibitive, approximate tensor-network methods typically manage complexity by compressing intermediate objects. On regular lattices, work such as tensor renormalization group (TRG) established that effective truncation of otherwise intractable intermediate states is essential to scalable approximate contraction [12]. More recently, these ideas have been extended from regular topologies to general graphs. Pan et al. proposed a general-purpose approximate contraction algorithm for arbitrary connected structures [7]; Gray and Chan incorporated approximation directly into contraction planning so that compression and contraction order can be co-optimized [8]; and Ma et al. developed a density-matrix-style approximation framework for general graphs [9]. These methods show that approximation on irregular graphs can be treated systematically. Our focus differs in that the final output tensor must still be fully produced, so the natural place for approximation is the internal separator state rather than the output boundary.

Randomized low-rank approximation and sketching. Our implementation also draws on randomized numerical linear algebra. Randomized low-rank approximation demonstrates that dominant subspaces can often be captured efficiently through random projections, followed by stable decomposition in a much smaller projected space, without first computing an expensive exact factorization [13]. More broadly, Count Sketch and Tensor Sketch illustrate how randomized linear maps can compress high-dimensional objects while preserving useful structure [14, 15]. Very recent work has begun to discuss sketching directly inside tensor network contraction [11]. In our setting, randomized compression is an implementation tool inside a separator-state framework; it does not change the problem definition.

Connections to elimination in graphical models. At the algorithmic level, our approach is also related to bucket elimination and mini-bucket methods in graphical models [16, 17]. Those methods emphasize that what must be propagated upward is not every local detail, but only the interface information that remains relevant to the rest of the graph. When the interface becomes too large, one needs a controlled approximation at the intermediate level. We revisit this viewpoint in the setting of open-leg tensor networks with full dense materialization, and we explicitly treat preservation of the output boundary as part of the method definition.

3 Method

3.1 Overview

Let $\mathcal{N}$ be a tensor network with open-leg set $\mathcal{O}$ and internal contraction label set $\mathcal{I}$. Contracting all internal labels yields the full output tensor

$$X = \mathrm{Mat}(\mathcal{N}) \in \mathbb{R}^{d_1 \times \cdots \times d_{|\mathcal{O}|}},$$

where Mat denotes dense materialization of the network. The end-to-end cost is

$$T_{\text{total}} = T_{\text{contract}} + T_{\text{emit}},$$

with T_{contract} the cost of internal contraction and approximation, and T_{emit} the cost of writing the result into the output buffer. In the full-materialization regime, T_{emit} is largely unavoidable, so our method targets T_{contract} by approximating only the separator states propagated inside the network, never the output boundary itself.

The pipeline has three main components:

1. build a binary partition tree on the interaction graph so that contraction on a general graph becomes a sequence of binary merges;

2. represent each subtree by a separator state that summarizes the information still visible to the rest of the network; and

3. recursively merge those separator states and recompress them after each merge to control intermediate growth.

3.2 Partition tree and separator states

To organize recursive contraction on a general graph, we construct a partition tree over the interaction graph of the tensor network. Each tree node v corresponds to a sub-network $\mathcal{N}_v$. Leaves correspond to original tensors, internal nodes correspond to merges of two child sub-networks, and the root corresponds to the full network.

For each node v, we track three sets of labels:

- **output labels** $\mathcal{O}_v$, which remain connected to the final output side;
- **boundary labels** $\mathcal{B}_v$, which connect the subtree to the rest of the network; and
- **cut labels** $\mathcal{C}_v$, which connect the left and right children and are eliminated at the current merge.

The root has no boundary labels because it is no longer connected to an exterior network, and leaves have no cut labels because they are not further split.

Given an output block Ω_b, we define the *separator state* at node v as

$$M_v \in \mathbb{R}^{d_{\text{open}}(v)\times d_{\text{bnd}}(v)},$$

where

$$d_{\text{open}}(v) = \prod_{\ell\in\mathcal{O}_v} d_\ell, \qquad d_{\text{bnd}}(v) = \prod_{\ell\in\mathcal{B}_v} d_\ell.$$

Here d_ℓ denotes the local dimension of label ℓ under the current output block. Rows of M_v index output-side configurations and columns index boundary-side configurations. Thus, M_v captures the local behavior of subtree $\mathcal{N}_v$ that is visible to its parent under block Ω_b.

Explicitly storing M_v is often too expensive, so we work with a low-rank factorization

$$M_v \approx A_v B_v^\top,$$

with

$$A_v \in \mathbb{R}^{d_{\text{open}}(v)\times r_v}, \qquad B_v \in \mathbb{R}^{d_{\text{bnd}}(v)\times r_v},$$

where r_v is the interface rank. This representation cleanly separates the output side from the boundary side and makes the approximation location explicit: the output boundary is preserved, while only the separator state is compressed.

3.3 Recursive merging on the tree

Once separator states are defined, the computation on the partition tree becomes a uniform recursion.

Leaf nodes. For a leaf node v, the sub-network contains only one original tensor. After restricting open labels to the current output block, we reshape the tensor into

$$T_v \in \mathbb{R}^{d_{\text{open}}(v)\times d_{\text{bnd}}(v)}.$$

Without approximation, T_v is the exact separator state. With approximation, we compress T_v into the factorized form $A_v B_v^\top$.

Internal nodes. Let the children of internal node v be v_L and v_R, with separator states

$$M_{v_L} \approx A_{v_L} B_{v_L}^\top, \qquad M_{v_R} \approx A_{v_R} B_{v_R}^\top.$$

The parent state is built in three steps.

(i) Merge the output side. Because

$$\mathcal{O}_v = \mathcal{O}_{v_L} \cup \mathcal{O}_{v_R}, \qquad \mathcal{O}_{v_L} \cap \mathcal{O}_{v_R} = \varnothing,$$

the output configuration space of the parent is the Cartesian product of the two child spaces. At the factor level, this corresponds to an outer-product combination, equivalently a Kronecker product,

$$A_{\text{mat}} = A_{v_L} \otimes A_{v_R},$$

whose intermediate column count is $r_{v_L} r_{v_R}$.

(ii) Contract cut labels and keep boundary labels. Let $\mathcal{B}_v$ denote the boundary labels retained by the parent and $\mathcal{C}_v$ the cut labels eliminated at the merge. The uncompressed boundary-side factor of the parent can be viewed as a contraction of the child boundary factors along $\mathcal{C}_v$:

$$B_{\text{mat}} = B_{v_L} \times_{\mathcal{C}_v} B_{v_R}.$$

Intuitively, this step eliminates the internal edges that should disappear at the current merge while preserving the labels that must remain visible to the parent.

(iii) Recompress. After the first two steps, the parent obtains an uncompressed representation

$$\widetilde{M}_v = A_{\text{mat}} B_{\text{mat}}^\top.$$

Because the intermediate rank has already expanded to $r_{v_L} r_{v_R}$, immediate recompression is necessary to prevent exponential rank growth along the tree. We therefore compress $\widetilde{M}_v$ back into the canonical separator-state form

$$M_v \approx A_v B_v^\top.$$

3.4 Adaptive rank selection

Recompression is required after every leaf construction and every internal merge. The simplest strategy is to use a fixed rank budget r for all nodes, but this implicitly assumes that separator states across the entire tree have comparable difficulty. In practice this assumption is rarely valid. Different nodes can have very different subtree sizes, boundary structures, local degrees of freedom under the current block, and spectral decay patterns. A uniform rank therefore tends to over-allocate easy nodes and under-allocate difficult ones.

We instead select the rank adaptively from a local error tolerance. Let

$$M \in \mathbb{R}^{m\times n}$$

be a matrix to compress, with singular value decomposition

$$M = U\Sigma V^\top, \qquad \Sigma = \mathrm{diag}(\sigma_1, \sigma_2, \ldots).$$

For its rank-r truncation

$$M_r = U_r \Sigma_r V_r^\top,$$

the relative Frobenius error is

$$\frac{\|M - M_r\|_F}{\|M\|_F} = \left(\frac{\sum_{i>r}\sigma_i^2}{\sum_i \sigma_i^2}\right)^{1/2}.$$

Given a tolerance τ, we choose the smallest r such that

$$\left(\frac{\sum_{i>r}\sigma_i^2}{\sum_i \sigma_i^2}\right)^{1/2} \le \tau.$$

In implementation, leaf nodes and merge nodes may use different tolerances, denoted by τ_{leaf} and τ_{merge}.

This formulation converts "how many channels to keep" into "how much relative energy may be discarded." Easy nodes with rapidly decaying spectra naturally stop at low rank, while difficult nodes retain more channels when necessary. The key advantage of adaptive rank selection is therefore not a larger average rank, but a better redistribution of rank budget according to local difficulty.

3.5 Implicit sketch-based merging

Adaptive rank selection alone does not remove the most important merge bottleneck. If the two children of an internal node have ranks r_L and r_R, then the merged interface naturally exhibits an intermediate rank product $r_L r_R$ before recompression. Explicitly constructing the large merged interface and then compressing it incurs the classic "inflate first, compress later" penalty. To avoid this, we introduce implicit sketch-based merging: instead of forming the full merged interface, we recover its dominant subspace via operator access [11, 13].

Let

$$\widetilde{M} \in \mathbb{R}^{m\times n}$$

be the interface to compress after merging. In the explicit path,

$$\widetilde{M} = A_{\mathrm{mat}} B_{\mathrm{mat}}^\top,$$

where A_{mat} comes from the output-side combination and B_{mat} from the contraction of boundary-side factors along the cut labels. Randomized compression does not require every entry of $\widetilde{M}$; it only needs access to the linear maps

$$x \mapsto \widetilde{M}x, \qquad y \mapsto \widetilde{M}^\top y.$$

Under our merge structure, these products can be evaluated as

$$\widetilde{M}x = A_{\mathrm{mat}}(B_{\mathrm{mat}}^\top x), \qquad \widetilde{M}^\top y = B_{\mathrm{mat}}(A_{\mathrm{mat}}^\top y),$$

which allows the entire compression step to be carried out through operator application without explicitly materializing $\widetilde{M}$.

We then apply a standard randomized SVD pipeline [13]. Draw a Gaussian test matrix

$$\Omega \in \mathbb{R}^{n\times \ell}, \qquad \Omega_{ij} \sim \mathcal{N}(0,1),$$

where $\ell = k + p$, with target rank estimate k and oversampling parameter p. Form the sample matrix

$$Y_0 = \widetilde{M}\Omega$$

through operator application. When spectral decay is slow, we optionally perform q power iterations:

$$Y_{t+1} = \widetilde{M}(\widetilde{M}^\top Y_t), \qquad t = 0, 1, \ldots, q-1.$$

After orthogonalization,

$$Q = \mathrm{orth}(Y_q) \in \mathbb{R}^{m\times \ell}$$

approximates the dominant column space of $\widetilde{M}$. We then project to a smaller matrix

$$M_{\mathrm{proj}} = Q^\top \widetilde{M} \in \mathbb{R}^{\ell\times n},$$

compute its exact SVD,

$$M_{\mathrm{proj}} = \widetilde{U}\Sigma V^\top,$$

and apply the same adaptive tail-energy rule to choose the retained rank r. Writing the truncated approximation as

$$\widetilde{M} \approx Q\widetilde{U}_r \Sigma_r V_r^\top,$$

we convert it back to our factorized separator-state form via

$$A = (Q\widetilde{U}_r)\Sigma_r^{1/2}, \qquad B = V_r \Sigma_r^{1/2},$$

so that

$$\widetilde{M} \approx AB^\top.$$

Adaptive rank selection determines how many channels to keep, while randomized sketching determines how to find those channels without explicitly constructing the inflated merge interface.

Algorithm 1 Full-materialization pipeline of AS-TNC

Input: Tensor network $\mathcal{N}$, output blocks $\{\Omega_b\}_{b=1}^{B}$, compression configuration θ
Output: Full output tensor X

```
Construct a partition tree T on the interaction graph of N
Initialize output buffer X
Initialize state cache C
for each output block Ω_b do
    S_root ← BuildState(root(T), Ω_b, θ, C)
    X[Ω_b] ← ReadoutRoot(S_root)
end for
return X

function BuildState(v, Ω_b, θ, C)
    if the state for (v, Ω_b, θ) is reusable from cache then
        return cached state
    end if
    if v is a leaf then
        Form local tensor T_v under block Ω_b
        Build separator state M_v on (O_v, B_v)
        Compress M_v if requested
        Cache and return the state
    else
        S_L ← BuildState(v_L, Ω_b, θ, C)
        S_R ← BuildState(v_R, Ω_b, θ, C)
        Merge the output-side factors of S_L and S_R
        Contract boundary-side factors along C_v
        Recompress the merged separator state
        Cache and return the state
    end if
end function

function ReadoutRoot(S_root)
    Contract the last rank dimension of the root state
    return dense output block for Ω_b
end function
```

3.6 Algorithm summary

Algorithm 1 summarizes the full pipeline. Output blocking organizes the dense-materialization workflow, the partition tree rewrites contraction on a general graph into recursive binary merges, separator states propagate local information upward, and recompression controls intermediate-state growth.

4 Experiments

We organize the experiments around three questions: Can AS-TNC deliver a useful accuracy-time trade-off for full-materialization TNC? Do the gains actually come from internal contraction rather than output write-back? And how do approximation strength and individual mechanisms affect the result?

4.1 Experimental setup

We report four representative classical networks: Ring-8 [18], Tree-8 [19], Grid 3×3 [1, 12], and Random-8 [20, 21]. Here "8" denotes the number of nodes and 3×3 denotes the two-dimensional grid size. Each node has one open leg, the physical dimension is fixed to 3, and the bond dimension is fixed to 4. We use AS-TNC-L1/L2/L3 to denote different approximation strengths; unless otherwise stated, AS-TNC refers to the medium-strength default preset. We compare exact contraction (exact-TNC), a fixed-rank baseline, and AS-TNC. The fixed-rank baseline uses rank 4 for all interfaces, serving as a clear small-rank fast-approximation reference. Additional details about network generation, tolerances, and output blocking are given in Section A.2.

4.2 Main results

Table 1 reports the core comparison across the four networks. The fixed-rank baseline is consistently very fast, but also severely distorted: its relative error ranges from 0.65 to 0.98 on all four cases. In contrast, the default AS-TNC configuration keeps the relative error below 4.81×10^{-3} in every case while still achieving speedups from 13.98× to 84.05× over exact-TNC. The most discriminative case is Grid 3×3: exact contraction takes 37.20 seconds, whereas AS-TNC reduces the total time to 0.4426 seconds with only 4.08×10^{-3} relative error. Across the four networks, AS-TNC attains a geometric-mean speedup of 42.53× with average relative error 2.68×10^{-3}.

4.3 Where do the gains come from?

We next examine whether the end-to-end gains are truly contraction-side. Table 2 provides clear evidence: across the four main-result networks, the default AS-TNC configuration has average ratio $t_{\text{contract}}/t_{\text{total}} = 0.9992$, while its average contraction speedup and total speedup are 52.07× and 52.03×, respectively. The two numbers are nearly identical. Hence the end-to-end acceleration is explained almost entirely by internal contraction rather than by output emission artifacts. For Grid 3×3, the contraction speedup is 84.36× and the total speedup is 84.33×, showing essentially no practical gap.

At the mechanism level, we use Grid 3×3 as the most discriminative case and isolate two key components, as shown in Table 3. Removing implicit sketch-based merging leaves the error essentially unchanged but makes the total time 6.97× worse than the default. This indicates that implicit sketching lowers internal merge cost without sacrificing accuracy. Removing output blocking increases the total time by 2.40× and also raises the error from 4.08×10^{-3} to 1.06×10^{-2}. For this harder network, implicit sketch-based merging is the dominant and most stable source of efficiency, while output blocking also provides a meaningful benefit.

Table 1: Main results on four representative networks. AS-TNC provides a strong accuracy-time trade-off compared with exact contraction and a fixed-rank approximation baseline.

Case	exact-TNC time (s)	fixed-rank rel. error	fixed-rank speedup	AS-TNC rel. error	AS-TNC speedup
Ring-8	3.0072	0.8058	123.39×	1.83×10^{-3}	56.31×
Tree-8	1.1068	0.6524	57.69×	4.81×10^{-3}	49.45×
Grid 3×3	37.1995	0.9789	566.07×	4.08×10^{-3}	84.05×
Random-8	0.7326	0.9283	21.72×	$< 10^{-12}$	13.98×
Summary	—	—	—	avg. $= 2.68\times10^{-3}$	gmean $= 42.53\times$

Table 2: Stage-time decomposition. The end-to-end gains of AS-TNC are almost entirely explained by contraction-side acceleration rather than output write-back.

Case	$t_{\text{contract}}/t_{\text{total}}$	$t_{\text{emit}}/t_{\text{total}}$	contraction speedup	total speedup
Ring-8	0.9993	0.000694	56.37×	56.33×
Tree-8	0.9985	0.001454	50.65×	50.58×
Grid 3×3	0.9997	0.000333	84.36×	84.33×
Random-8	0.9993	0.000722	16.92×	16.91×
Average	0.9992	0.000801	52.07×	52.03×

Table 3: Mechanism evidence on Grid 3×3. Implicit sketch-based merging is the dominant efficiency source, while output blocking also matters.

Setting	rel. error	total time (s)	time ratio vs. default
AS-TNC (default)	4.08×10^{-3}	0.4426	1.00
no-blockwise	1.06×10^{-2}	1.0635	2.40
no-implicit	4.08×10^{-3}	3.0838	6.97

4.4 Approximation-strength control

We finally test whether approximation strength τ provides stable control over error. Table 4 shows a clear monotonic trend from L1 to L2 to L3. On Ring-8, the relative error increases from $< 10^{-12}$ to 1.83×10^{-3} and then to 1.55×10^{-2}. On Grid 3×3, it increases from $< 10^{-12}$ to 4.08×10^{-3} and then to 1.81×10^{-2}. On Random-8, L1 and L2 are effectively exact, while L3 rises to 1.26×10^{-2}. The approximation level thus acts as a practical error knob for AS-TNC.

5 Conclusion

We studied dense full materialization for tensor networks with open legs. Unlike conventional approximate contraction, which typically targets scalar outputs or local queries, our setting requires the final result to be emitted explicitly as a dense tensor. Under this requirement, the output representation cannot be changed, so the natural place for approximation is the internal contraction process rather than the output side. Based on this view, we introduced AS-TNC.

AS-TNC organizes full materialization as a unified pipeline: it partitions the output index space, builds a partition tree on the interaction graph, recursively propagates separator states while keeping the output boundary explicit, and suppresses interface growth through adaptive low-rank compression and implicit sketch-based merging. The method therefore approximates not the final output object, but the internal separator states that must be traversed in order to produce it.

Table 4: Sensitivity to approximation strength. Stronger approximation leads to larger relative error in a consistent way.

Case	L1 rel. error	L2 rel. error	L3 rel. error
Ring-8	$< 10^{-12}$	1.83×10^{-3}	1.55×10^{-2}
Grid 3×3	$< 10^{-12}$	4.08×10^{-3}	1.81×10^{-2}
Random-8	$< 10^{-12}$	$< 10^{-12}$	1.26×10^{-2}
Average rel. error	$< 10^{-12}$	1.97×10^{-3}	1.54×10^{-2}

Experiments on Ring-8, Tree-8, Grid 3×3, and Random-8 show that AS-TNC delivers a clear and usable accuracy-time trade-off, achieving substantial speedups over exact contraction while keeping relative error small. Stage-wise decomposition shows that the gains come almost entirely from the contraction stage. Mechanism analysis identifies implicit sketch-based merging as the most important and stable efficiency source, with output blocking also contributing on harder networks. Strength scans further show that the approximation level provides an effective knob for balancing accuracy and runtime.

More broadly, the results support a simple message: when the dense output is part of the task definition, the right target for approximation is not the output tensor itself, but the internal separator states propagated during contraction. Along this direction, partition-tree recursion, separator-state representation, and sketch-driven compression can be combined into a practical framework for open-leg tensor network contraction with full materialization.

References

[1] Román Orús. A practical introduction to tensor networks: Matrix product states and projected entangled pair states. *Annals of Physics*, 349:117–158, 2014. doi: 10.1016/j.aop.2014.06.013.

[2] Igor L. Markov and Yaoyun Shi. Simulating quantum computation by contracting tensor networks. *SIAM Journal on Computing*, 38(3):963–981, 2008. doi: 10.1137/050644756.

[3] Eugene F. Dumitrescu, Allison L. Fisher, Timothy D. Goodrich, Travis S. Humble, Blair D. Sullivan, and Andrew L. Wright. Benchmarking treewidth as a practical component of tensor network simulations. *PLOS ONE*, 13(12):e0207827, 2018. doi: 10.1371/journal.pone.0207827.

[4] Jonathan Jakes-Schauer, David Anekstein, and Pawel Wocjan. Carving-width and contraction trees for tensor networks. *arXiv preprint arXiv:1908.11034*, 2019.

[5] Johnnie Gray and Stefanos Kourtis. Hyper-optimized tensor network contraction. *Quantum*, 5:410, 2021. doi: 10.22331/Q-2021-03-15-410.

[6] Cupjin Huang, Fang Zhang, Michael Newman, Xiaotong Ni, Dawei Ding, Junjie Cai, Xun Gao, Tenghui Wang, Feng Wu, Gengyan Zhang, et al. Efficient parallelization of tensor network contraction for simulating quantum computation. *Nature Computational Science*, 1:578–587, 2021. doi: 10.1038/s43588-021-00119-7.

[7] Feng Pan, Pengfei Zhou, Sujie Li, and Pan Zhang. Contracting arbitrary tensor networks: General approximate algorithm and applications in graphical models and quantum circuit simulations. *Physical Review Letters*, 125(6):060503, 2020. doi: 10.1103/PhysRevLett.125.060503.

[8] Johnnie Gray and Garnet Kin-Lic Chan. Hyperoptimized approximate contraction of tensor networks with arbitrary geometry. *Physical Review X*, 14(1):011009, 2024. doi: 10.1103/PhysRevX.14.011009.

[9] Linjian Ma, Matthew T. Fishman, E. Miles Stoudenmire, and Edgar Solomonik. Approximate contraction of arbitrary tensor networks with a flexible and efficient density matrix algorithm. *Quantum*, 8:1580, 2024. doi: 10.22331/q-2024-12-27-1580.

[10] Rina Dechter. Bucket elimination: A unifying framework for probabilistic inference. In *Learning in graphical models*, pages 75–104. Springer, 1998.

[11] Mike Heddes, Igor Nunes, Tony Givargis, and Alex Nicolau. Approximating tensor network contraction with sketches. *arXiv preprint arXiv:2603.07387*, 2026.

[12] Michael Levin and Cody P. Nave. Tensor renormalization group approach to two-dimensional classical lattice models. *Physical Review Letters*, 99(12):120601, 2007. doi: 10.1103/PhysRevLett.99.120601.

[13] Nathan Halko, Per-Gunnar Martinsson, and Joel A. Tropp. Finding structure with randomness: Probabilistic algorithms for constructing approximate matrix decompositions. *SIAM Review*, 53(2):217–288, 2011. doi: 10.1137/090771806.

[14] Moses Charikar, Kevin Chen, and Martin Farach-Colton. Finding frequent items in data streams. *Theoretical Computer Science*, 312(1):3–15, 2004. doi: 10.1016/S0304-3975(03)00400-6.

[15] Ninh Pham and Rasmus Pagh. Fast and scalable polynomial kernels via explicit feature maps. In *Proceedings of the 19th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 239–247, 2013. doi: 10.1145/2487575.2487591.

[16] Rina Dechter. Bucket elimination: A unifying framework for probabilistic inference. In *Proceedings of the Twelfth Conference on Uncertainty in Artificial Intelligence*, pages 211–219, 1996.

[17] Rina Dechter and Irina Rish. Mini-buckets: A general scheme for generating approximations in automated reasoning. In *Proceedings of the Fifteenth International Joint Conference on Artificial Intelligence*, pages 1297–1303, 1997.

[18] J. A. Bondy and U. S. R. Murty. *Graph Theory*. Springer, 2008.

[19] Reinhard Diestel. *Graph Theory*. Springer, 5 edition, 2017.

[20] Paul Erdős and Alfréd Rényi. On random graphs i. *Publicationes Mathematicae (Debrecen)*, 6:290–297, 1959.

[21] Edgar N. Gilbert. Random graphs. *The Annals of Mathematical Statistics*, 30(4):1141–1144, 1959. doi: 10.1214/aoms/1177706098.

A Appendix

A.1 Implementation details

Partition-tree construction. Let the interaction graph of the tensor network be $G = (V, E, w)$, where each node corresponds to one original tensor and edges connect tensors that share internal labels. In the current implementation, edge weights accumulate the dimensions contributed by shared internal labels. On each induced subgraph $G[U]$, we recursively perform a bipartition: if the subgraph is connected, we use a Stoer–Wagner minimum cut to obtain U_L and U_R; if the subgraph is degenerate or edgeless, we fall back to a "single node versus the rest" split. This process constructs a binary partition tree bottom-up.

Output blocking. Let the global open-label order be $\mathcal{O} = (\ell_1, \ldots, \ell_m)$. In the implementation, only the first $q =$ `block_label_count` open labels are block-partitioned. For each blocked label ℓ_i, its index set is divided into contiguous chunks of length `chunk_size`:

$$I_{\ell_i}^{(1)}, I_{\ell_i}^{(2)}, \ldots.$$

An output block Ω_b is then the Cartesian product of the selected local chunks. Blocking is therefore not only a scheduling device for write-back; it also changes the local output dimensions seen by each subtree under the current block.

Local block keys and cross-block caching. For a tree node v and output block Ω_b, we define the local block key

$$\kappa_v(\Omega_b) = \left\{ \begin{array}{l} (\ell, I_{b,\ell}) : \ell \in \mathcal{O}_v, \\ \ell \text{ is sliced in the current block} \end{array} \right\}.$$

This key retains only the slice constraints visible to subtree v and ignores unrelated block information. The cache key can therefore be written as

$$K(v, \Omega_b) = \big(\text{node_key}(v),\ \kappa_v(\Omega_b),\ \text{sig}(\theta)\big),$$

where $\text{sig}(\theta)$ is the signature of the compression configuration, including rank-selection rules, tolerances, randomized parameters, and optimization strategy. If two global output blocks induce the same $\kappa_v(\Omega_b)$ at node v, the cached state can be reused directly.

Root readout. At the root, the boundary label set is empty, i.e.,

$$\mathcal{B}_{\text{root}} = \varnothing.$$

The root state therefore degenerates from a general output-side times boundary-side interface into the dense result for the current output block. If the root state is written as $M_{\text{root}} \approx A_{\text{root}} B_{\text{root}}^{\top}$, the block output is obtained by contracting the final rank dimension.

Exact-contraction baseline. The exact baseline (exact-TNC) shares the same tensor-network object and the same output-label order as AS-TNC. For each output block Ω_b, it directly performs an exact full contraction,

$$X_{\Omega_b} = \text{contract_full}(\mathcal{N}, \Omega_b),$$

using the exact contraction interface of `opt_einsum` under the chosen optimization strategy. Thus, the difference between AS-TNC and exact-TNC lies primarily in the internal computation path rather than the output interface.

Reproducibility. In randomized compression, the current implementation uses a deterministic pseudorandom generator seeded jointly by a base random seed and the state key. As a result, the randomized compression outcome is reproducible for the same network, output block, tree node, and configuration.

A.2 Experimental details

Network generation. The main experiments use Ring-8, Tree-8, Grid 3×3, and Random-8. Tree-8 is generated using a random labeled tree (`networkx.random_labeled_tree`) with 8 nodes. Random-8 uses an Erdős–Rényi random graph with a connectedness check (`networkx.erdos_renyi_graph`), again with 8 nodes and edge probability $p = 0.35$. Both are driven by global random seeds $\{0, 1, 2\}$ in the main experiments.

Tolerance presets. Leaf compression and merge compression use separate tolerances τ_{leaf} and τ_{merge}. The three presets used in the strength scan are

$$\begin{array}{ll} \text{L1} & (5.0 \times 10^{-4},\ 2.0 \times 10^{-3}), \\ \text{L2} & (1.0 \times 10^{-3},\ 5.0 \times 10^{-3}), \\ \text{L3} & (3.0 \times 10^{-3},\ 1.5 \times 10^{-2}). \end{array}$$

In addition, AS-TNC uses a `depth_open`-style schedule in which the local tolerance is adapted as

$$\tau_{\text{local}} = \tau_{\text{base}} \cdot (1.5)^{\text{depth}} / (\max(1, \text{open_count}))^{0.5}.$$

Output-blocking policy. All main experiments enable output blocking with `block_labels=2` and `chunk_size=1`. That is, two open-leg labels are selected as blocking axes and sliced one element at a time, so the full output tensor is contracted and written back block by block.

Timing protocol. We use a unified timing protocol with the same task, the same block schedule, and the same stage decomposition for all methods. For each network instance, we first generate the same set of output blocks and then run both exact-TNC and the approximate methods on that schedule. Timing uses a high-resolution

clock and accumulates contraction time and emit time separately for every output block, ultimately reporting

$$T_{\text{total}} = T_{\text{contract}} + T_{\text{emit}}.$$

We also report $t_{\text{contract}}/t_{\text{total}}$ as a stage-share metric. Speedup is defined as

$$\text{speedup} = T_{\text{exact-TNC}}/T_{\text{method}},$$

and results are aggregated across random seeds $\{0, 1, 2\}$ to reduce single-run noise.

A.3 Supplementary visual summary

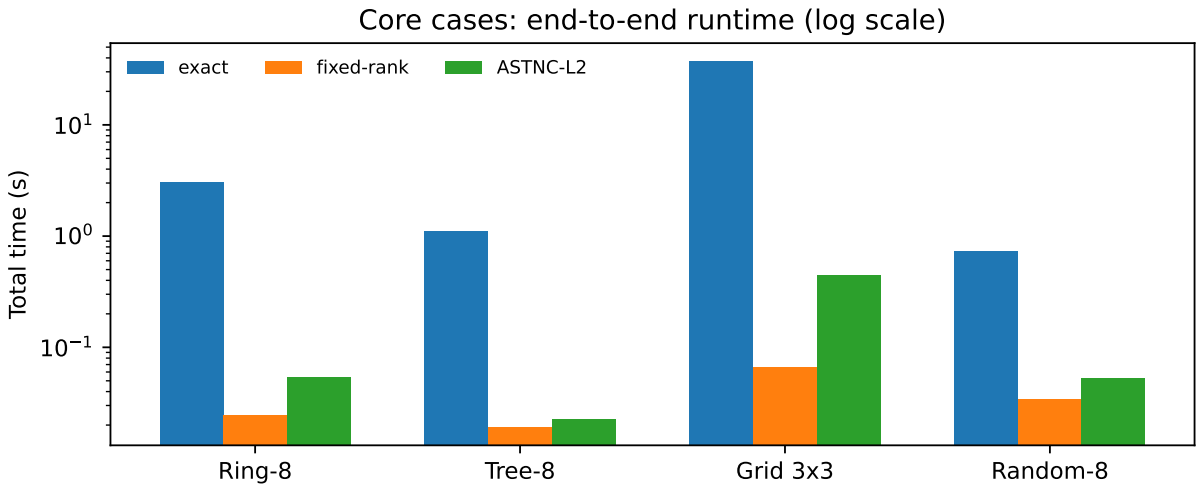

Figure 1: Supplementary runtime overview across the main benchmark cases. The figure is reused from the original `paper` assets and can be included directly in an arXiv upload.

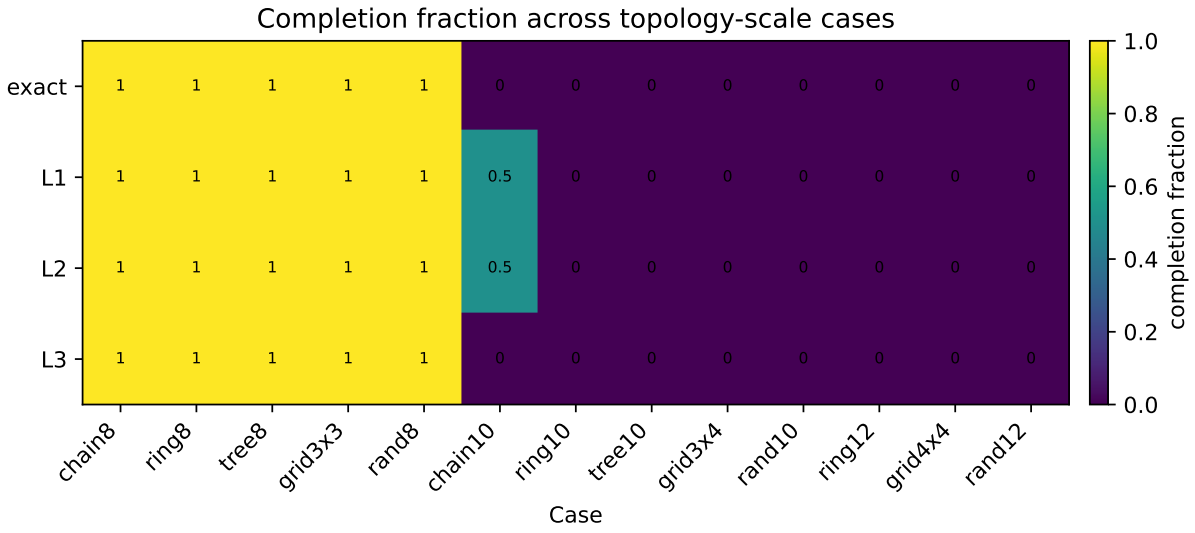

Figure 2: Supplementary regime-status summary across topologies and scales. This figure provides a broader context for the easy, medium, and boundary regimes discussed in the original drafting materials.